FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 8. Personalización de Git

Configuración, atributos y ganchos (hooks)

Git se adapta a la forma de trabajar de cada persona y equipo. Este capítulo explica las opciones de configuración más útiles, los atributos por tipo de archivo y los ganchos que automatizan validaciones.

8.1 Configuración de Git

Además de `user.name` y `user.email`, hay muchas opciones útiles. Se consultan con `git config --list` y se editan con `git config`.

```
git config --global core.editor "code --wait"
git config --global commit.template ~/.gitmessage.txt
git config --global core.pager "less -FRX"
git config --global core.excludesfile ~/.gitignore_global
git config --global help.autocorrect 20
git config --global color.ui auto
git config --global user.signingkey <ID_CLAVE_GPG>
```

La plantilla de commit (`commit.template`) es práctica para recordar al equipo el formato esperado de los mensajes. `core.excludesfile` define un `.gitignore` global (por ejemplo, para archivos del editor).

8.2 Atributos de Git

El archivo `.gitattributes` define comportamientos por tipo de archivo o ruta:

- **Archivos binarios:** `*.png binary` evita intentar mostrar diferencias de texto; `*.docx diff=word` permite usar un conversor para compararlos.
- **Fin de línea:** `* text=auto` normaliza los saltos de línea entre sistemas operativos.
- **Exportación:** `export-ignore` excluye archivos al crear un archivo con `git archive`.
- **Estrategias de fusión:** `merge=ours` para conservar siempre la versión propia en ciertos archivos.

```
# .gitattributes
* text=auto
*.pdf binary
*.docx binary
```

8.3 Ganchos de Git (hooks)

Los ganchos son scripts que Git ejecuta automáticamente ante ciertos eventos. Viven en `.git/hooks/` y deben ser ejecutables.

- **Del lado del cliente:** `pre-commit` (revisar antes de confirmar), `prepare-commit-msg`, `commit-msg` (validar el mensaje), `post-commit`, `pre-push`.
- **Del lado del servidor:** `pre-receive`, `update` y `post-receive`, útiles para imponer políticas, rechazar pushes o notificar al equipo.

```
#!/bin/sh
# .git/hooks/commit-msg: exige al menos 10 caracteres
if [ $(wc -c < "$1") -lt 10 ]; then
  echo "Mensaje de commit demasiado corto"
  exit 1
fi
```

8.4 Ejemplo de política aplicada con Git

Un equipo puede combinar un gancho `pre-receive` que rechace pushes con mensajes sin número de incidencia, ramas protegidas en GitHub y una plantilla de commit para mantener un historial coherente y trazable.

Conclusión del capítulo

Personalizar Git reduce errores y fricción: unas pocas líneas de configuración, atributos y ganchos pueden estandarizar el trabajo de todo el equipo.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2